Practice Exam Cheat Sheet

This guide is a summary of the key concepts and code patterns you've learned. Use it as a reference to help you assemble the tools needed to solve the practice exam tasks.

1. Core Socket Concepts

Remember to `import socket` for all of these.

TCP Server

The lifecycle of a TCP server that waits for clients to connect.

```
# 1. Create the socket
server_sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# 2. Bind it to an address to listen
server_sock.bind(('localhost', 10001))

# 3. Put it in listening mode
server_sock.listen(5)

# 4. Wait for a client to connect (this blocks!)
# IMPORTANT: .accept() returns a NEW socket for this one client.
connection, client_addr = server_sock.accept()
```

TCP Client

The lifecycle of a TCP client that initiates a connection.

```
# 1. Create the socket
client_sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# 2. Connect to the server
client_sock.connect(('localhost', 10001))
```

TCP Communication (after connection)

- **Sending Data:** `sendall` is reliable for TCP.

  ```
  sock.sendall(b"some message")
  ```

- **Receiving Data:** `recv` waits for data. If it returns empty bytes (`b''`), the other side has closed the connection.

```
data = sock.recv(1024)
if not data:
    # The client has disconnected.
```

UDP Communication

UDP is connectionless. You specify the destination for every message.

```
# Create a UDP socket
udp_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

# To send data (as a client)
archive_addr = ('localhost', 10002)
udp_sock.sendto(b"Search", archive_addr)

# To receive data (as a client or server)
# Returns the data AND the sender's address
data, sender_addr = udp_sock.recvfrom(1024)

# For a UDP server, you must also bind it
udp_sock.bind(('localhost', 10002))
```

---

2. Concurrency with `select`

This is the pattern for handling multiple clients at the same time. Remember to `import select` .

```
# 1. Setup: Start with a list containing only your main listening socket
inputs = [server_sock]

# 2. The Loop:
while True:
    # 3. Wait: select() blocks until a socket is ready to be read
    readables, _, _ = select.select(inputs, [], [])

    # 4. Process: Loop through the sockets that are ready
    for s in readables:
        # 5. Is it a NEW connection?
        if s is server_sock:
            # Accept the new client and ADD it to the inputs list to be watched
            connection, client_addr = server_sock.accept()
            inputs.append(connection)

        # 6. Or is it an EXISTING client sending a message?
        else:
            # Receive data from the client socket 's'
            data = s.recv(1024)

            # IMPORTANT: Handle disconnection!
            if not data:
                # Remove the socket from the inputs list and close it
                inputs.remove(s)
                s.close()
            else:
                # This is where you put your logic to handle the client's message
                # (e.g., if data == b"I would like a task"...)
```

---

3. Command-Line Arguments

Your server needs to accept parameters from the command line. Remember to `import sys`.

```python
# Check if the correct number of arguments was given
if len(sys.argv) != 4:
    print("Usage: python3 my_server.py <ip> <port> <num_students>")
    sys.exit(1)

# Access the arguments
listen_ip = sys.argv[1]
listen_port = int(sys.argv[2])  # Don't forget to convert numbers to int()!
num_students = int(sys.argv[3])
```

---

4. Data Handling: Bytes vs. Strings

- **To send a string:** You must `.encode()` it first.

  ```python
  my_string = "Here is the task!"
  sock.sendall(my_string.encode())
  ```

- **When you receive data:** It will be in `bytes`. To print it or check its content as a string, you must `.decode()` it.

  ```python
  data = sock.recv(1024)
  print(data.decode())
  ```

- **Direct Bytes:** You can also define bytes directly with a `b` prefix.

  ```python
  if data == b"Thank you":
      sock.sendall(b"You're welcome")
  ```

Good luck